

Final Overall Workflow + Dynamic Smoke-Test - Overview Truth

Canonical workflow map for skeleton batches, organ batches, dynamic idempotent smoke checks, companion updates, and final operator/config integration.

1. Canonical truth model

Area	Canonical rule
Shared project root	/workspace is the shared project workspace for code, runs, artifacts, models, outputs, and generated platform files.
Skeleton evidence	/mnt/egress/dev-recordings/skeleton/<batch-slug>/ with POSTCHECK.md and INTEGRATION_REQUEST.md.
Organ evidence	/mnt/egress/organs/dev-recordings/organs/<batch-slug>/ with POSTCHECK.md and INTEGRATION_REQUEST.md.
Skeleton companion	/mnt/ingress/infra/skeleton/companion/<batch-slug>/COMPANION.md.
Organ companion	/mnt/ingress/infra/organs/companion/<batch-slug>/COMPANION.md.
Config edits	Never edit config/lv/workflow during skeleton or organ implementation batches. Only vmuser/operator config-integration batches may do that.
Missing files	Stop, list exact missing files, classify required/optional, and do not invent evidence.
Live actions	Smoke tests must be safe and idempotent. No terraform apply, no kubernetes destructive change, no live RunPod creation, no real organ action unless explicitly gated.

2. Overall order of work

Phase	Do this	When smoke runs	When companion updates	Who changes config
1	Run all skeleton batches first: skeleton 01, skeleton 02, ... skeleton 24.	After every skeleton batch and again at skeleton-complete.	After logical groups, after contract changes, or after skeleton-complete.	Nobody. Config is not edited here.
2	Generate/update skeleton companions as you go or after logical groups.	Use the latest smoke report as input to companion update.	S-T6. Companion documents checked skeleton state.	Nobody.
3	Then run organ batches: organ R01, organ R02, ... organ R12.	After every organ batch and again at organ-complete.	After organ logical groups, after safety/output contract changes, or after organ-complete.	Nobody.
4	Do final operator/config integration from completed evidence.	Run pre-config before integration and post-config after integration.	Use final companions as evidence.	Only vmuser/operator config-integration batch edits config/lv/workflow.

3. Dynamic idempotent smoke-test architecture

Layer	Path	Purpose	Created/updated by
Protocol/spec	/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md	Defines safe dynamic smoke protocol, module contract, PASS/WARN/FAIL rules, and forbidden actions.	ChatGPT drafts; Codex may write to docs.
Orchestrator	/workspace/scripts/smoke_current_state.sh	Discovers and runs applicable smoke modules for current phase and platform state.	Codex writes/updates from the spec.
Smoke modules	/workspace/tests/smoke.d/*.smoke.sh	Specific safe checks for kubernetes, terraform, agentfield, runpod, grn, skeleton, organs, config.	Codex adds modules as platform capabilities expand.
Reports	/workspace/runs/smoke/<timestamp-phase>/SMOKE_REP.ORT.md	Current evidence: what ran, what passed/warned/failed, skipped optional checks, exact commands.	Generated by orchestrator.

Smoke command pattern

```
BATCH_SLUG="01-runtime-substrate" bash /workspace/scripts/smoke_current_state.sh skeleton-progress
BATCH_SLUG="R01-real-organ-foundation" bash /workspace/scripts/smoke_current_state.sh organ-progress
bash /workspace/scripts/smoke_current_state.sh skeleton-complete
bash /workspace/scripts/smoke_current_state.sh organ-complete
bash /workspace/scripts/smoke_current_state.sh pre-config
bash /workspace/scripts/smoke_current_state.sh post-config
```

4. Skeleton timeline with file truth and smoke placement

Step	Owner	Action	Files ChatGPT reads/uploads	Files ChatGPT creates	Codex must access/run/create
S-T1	ChatGPT	Create next skeleton batch package.	general_new_chat_batch_generation_prompt.md; 00_skeleton_dummy_master_implementation_companion.md; skeleton_dummy_codex_batch_plan.md; CODEX_PROMPT.txt; PROJECT_CACHE.md; SPEC.md; RUN_INSTRUCTIONS.md; POSTCHECK_TEMPLATE.md; CONFIG_TOOL.md; optional latest companion/smoke/evidence.	codex_skeleton_batch_<N>_<slug>.zip; batch-specific CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md.	No Codex run yet.
S-T2	User/Codex	Put batch into project/Codex context.	No new ChatGPT upload required unless batch needs repair.	None.	Project has /workspace, generated batch zip/files, recording rules.
S-T3	Codex as researchscientist	Implement skeleton batch. Prompt should name batch files and tell Codex to follow them; do not paste their full internal prompts.	Usually none.	None unless repair prompt needed.	Read CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, CODEX_RECORDING_INSTRUCTIONS.md. Create/update project code under /workspace and evidence under /mnt/egress/dev-recordings/skeleton/<batch-slug>/.
S-T4	Codex	Run dynamic smoke after this batch.	None.	None.	Run BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh skeleton-progress. Creates /workspace/runs/skeleton/.../SMOKE_REPORT.md.
S-T5	User/ChatGPT/Codex	Read smoke result and decide PASS/WARN/FAIL before continuing.	Latest SMOKE_REPORT.md; latest POSTCHECK.md; latest INTEGRATION_REQUEST.md if analysis is needed.	Optional missing-file report or repair prompt.	Codex fixes only if asked. Do not continue to next batch on FAIL.
S-T6	ChatGPT, optional Codex write	Update skeleton companion after a checked logical group or contract change.	COMPANION_GENERATOR_INSTRUCTIONS.md; latest skeleton POSTCHECK.md; INTEGRATION_REQUEST.md; SMOKE_REPORT.md; codebase analysis; existing skeleton COMPANION.md/INDEX.md.	Updated COMPANION.md content; optional INDEX.md update; missing-file report.	Codex may write provided content to /mnt/ingress/infra/skeleton/companion/<batch-slug>/COMPANION.md. No config edits.
S-T7	Repeat	Continue S-T1 to S-T6 until skeleton 24 complete.	Same as above, using current batch files and prior evidence.	Next batch packages and companion updates.	Same as above.
S-T8	Codex + ChatGPT	Run skeleton-complete smoke and update final skeleton companion/index.	Final skeleton evidence, all key companions, final SMOKE_REPORT.md.	Final skeleton companion summary; optional INDEX.md.	Run bash /workspace/scripts/smoke_current_state.sh skeleton-complete.
S-T9	ChatGPT	Prepare skeleton evidence for later config integration; do not edit config.	Skeleton POSTCHECK.md files; INTEGRATION_REQUEST.md files; companion docs; smoke reports.	Skeleton-side evidence summary or manifest input notes.	No config run yet.

Skeleton logical groups for S-T6 companion updates

Group	Update companion when this group reaches checked state
Foundation/runtime	Batch creates/changes package layout, CLI entrypoints, shared workspace paths, or evidence roots.
Dummy science contracts	Batch creates/changes dummy outputs, JSON/Markdown schemas, mechanism report names, or run artifacts.
Pipeline/orchestration	Batch creates/changes staged runs, manifests, dry-run runners, or batch execution rules.
Evaluation/reporting	Batch creates/changes validation, metrics, dashboards, postcheck output, or report schemas.
Integration bridge	Batch creates/changes INTEGRATION_REQUEST.md semantics, config handoff details, or companion requirements.

5. Organ timeline with file truth and smoke placement

Step	Owner	Action	Files ChatGPT reads/uploads	Files ChatGPT creates	Codex must access/run/create
O-T1	ChatGPT	Create next organ batch only after relevant skeleton contract is stable.	general_new_chat_organ_batch_generation_prompt.md; 00_transition_to_real_organ_master.md; transition_real_organ_codex_batch_plan.md; CODEX_PROMPT.txt; PROJECT_CACHE.md; SPEC.md; RUN_INSTRUCTIONS.md; POSTCHECK_TEMPLATE.md; CONFIG_TOOL.md; relevant skeleton COMPANION.md/contract summary; optional latest organ evidence.	codex_organ_batch_<N>_<slug>.zip; organ batch-specific CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md.	No Codex run yet.
O-T2	User/Codex	Put organ batch into project/Codex context.	No new ChatGPT upload required unless batch needs repair.	None.	Project has /workspace, generated organ batch zip/files, organ recording rules.
O-T3	Codex as researchscientist	Implement organ batch safely. Prompt should name the batch files and the relevant skeleton companion contract.	Usually none.	None unless repair prompt needed.	Read CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, CODEX_ORGAN _RECORDING_INSTRUCTIONS.md, skeleton companion/contract. Create/update code under /workspace and evidence under /mnt/egress/orga ns/dev-recordings/organs/<batch-slug>.
O-T4	Codex	Run dynamic smoke after this organ batch.	None.	None.	Run BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh organ-progress. Creates /workspace/runs/smoke/ .../SMOKE_REPORT.md.
O-T5	User/ChatGPT/Codex	Read smoke result and decide PASS/WARN/FAIL before continuing.	Latest organ SMOKE_REPORT.md; latest POSTCHECK.md; latest INTEGRATION_REQUEST.md; relevant skeleton contract if needed.	Optional missing-file report or repair prompt.	Codex fixes only if asked. Do not continue to next organ on FAIL.
O-T6	ChatGPT, optional Codex write	Update organ companion after checked organ state/logical group.	ORGAN_COMPANION_GENERATOR_INSTRUCTIONS.md; latest organ POSTCHECK.md; INTEGRATION_REQUEST.md; SMOKE_REPORT.md; codebase analysis; existing organ COMPANION.md/INDEX.md; relevant skeleton COMPANION.md.	Updated organ COMPANION.md content; optional INDEX.md update; missing-file report; safety-gate checklist.	Codex may write provided content to /mnt/ingress /infra/organs/companion/<batch-slug>/COMPANI ON.md. No config edits.
O-T7	Repeat	Continue O-T1 to O-T6 until organ R12 complete.	Same as above, using current organ batch files and accumulated skeleton/organ evidence.	Next organ batch packages and companion updates.	Same as above.
O-T8	Codex + ChatGPT	Run organ-complete smoke and update final organ companion/index.	Final organ evidence, all key companions, final SMOKE_REPORT.md, skeleton contract summary.	Final organ companion summary; optional INDEX.md.	Run bash /workspace/scripts/smoke_current_state.sh organ-complete.
O-T9	ChatGPT + Codex as vmuser/operator	Create INTEGRATION_MANIFEST.md and run dedicated config-integration batch.	INTEGRATION_REQUEST.md files; POSTCHECK.md files; COMPANION.md files; smoke reports; CONFIG_TOOL.md; operator integration templates.	INTEGRATION_MANIFEST.md; config-integration CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md.	Codex vmuser/operator reads generated config-integration batch, runs pre-config smoke, edits config/lv/workflows only as specified, then runs post-config smoke.

Organ logical groups for O-T6 companion updates

Group	Update companion when this group reaches checked state
Real-organ foundation	Batch connects real-organ modules while preserving skeleton output contracts.
Science engine modules	Batch changes GRN/NCA/diffusion/simulator behavior, parameters, schemas, or dry-run outputs.
External platform adapters	Batch adds Kubernetes, Terraform, RunPod, storage, queue, or agentfield adapters.
Safety and gating	Batch changes live-action guards, dry-run/live split, secrets handling, or operator approval rules.
Evidence/config handoff	Batch changes integration request semantics, smoke modules, status commands, or config needs.

6. Dynamic smoke modules: what they may consume and run

Module example	Consumes	Safe commands allowed	Must not do
00-core.smoke.sh	/workspace layout; Python/package metadata; git status if available.	test -d; python --version; import checks; file existence checks.	No deletes; no installs unless explicitly allowed.
10-skeleton.smoke.sh	Skeleton SPEC/RUN_INSTRUCTIONS; skeleton outputs; /mnt/egress/dev-recordings/skeleton/...	Skeleton dry-run/dummy commands; schema/file checks; metadata checks.	No config edits; no live organs.
20-organs.smoke.sh	Organ SPEC/RUN_INSTRUCTIONS; skeleton companion contract; organ outputs.	Organ dry-run commands; contract-preserving output checks.	No guarded-real/live actions unless explicitly requested.
30-grn.smoke.sh	GRN module/config files, safe sample inputs, expected artifact contract.	Unit-level or dry-run GRN commands; no expensive training by default.	No long GPU jobs; no external live calls unless gated.
40-kubernetes.smoke.sh	Kubernetes manifests, kube config if present, namespace context.	kubectrl version --client; kubectrl config current-context; kubectrl diff --server-side only if safe and configured.	No kubectrl apply/delete/scale/rollout restart.
50-terraform.smoke.sh	Terraform directories/modules/providers, tfvars examples if safe.	terraform fmt -check; terraform init -backend=false; terraform validate.	No terraform apply/destroy; no remote state mutation.
60-agentfield.smoke.sh	AgentField configs, local manifests, dry-run commands.	AgentField validate/status/dry-run checks.	No production agent launch.
70-runpod.smoke.sh	RunPod config templates, credentials presence check only, dry-run wrappers.	Validate config shape; check env var presence without printing secrets.	No pod creation, billing actions, live jobs.
90-config.smoke.sh	Config/lv/workflow files, integration manifest, generated operator batch.	pre-config read-only checks; post-config status/health checks.	No config edits outside operator integration step.

7. Final config integration truth

Step	Owner	Inputs	Outputs	Smoke
Pre-config readiness	ChatGPT + Codex vmuser/operator	Completed INTEGRATION_REQUEST.md files, POSTCHECK.md files, COMPANION.md files, SMOKE_REPORT.md files, CONFIG_TOOL.md.	Readiness report or missing-file list.	bash /workspace/scripts/smoke_current_state.sh pre-config
Manifest creation	ChatGPT	All completed skeleton/organ evidence and companion docs.	INTEGRATION_MANIFEST.md or manifest slice; config-integration batch files.	No smoke required inside ChatGPT; cite evidence in the manifest.
Config integration execution	Codex vmuser/operator	Generated config-integration CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, INTEGRATION_MANIFEST.md.	Config steps, lv profiles, aliases, health checks, bootstrap rows, integration POSTCHECK.md.	Run pre-config before, post-config after.
Post-config verification	Codex vmuser/operator + user	Updated config/lv/workflow files and generated integration evidence.	Final SMOKE_REPORT.md and config integration POSTCHECK.md.	bash /workspace/scripts/smoke_current_state.sh post-config

8. The one-page operating rule

ChatGPT creates batch plans, companion text, and integration manifests from named evidence files. Codex executes named batch files and exact smoke commands. Skeleton and organ Codex batches write code and evidence only. Dynamic smoke modules expand with the platform and stay safe/idempotent. Only the final vmuser/operator config-integration batch edits config/lv/workflow, and it is bracketed by pre-config and post-config smoke.